Simulación y control de un UR5 con gripper en ROS 2: construcción de paquetes, planificación de trayectorias con MoveIt y rutina Pick and Place

Samuel Jerónimo Moreno Mora
Robótica (RBTC)
Escuela Colombiana de Ingeniería Julio Garavito, Bogotá, Colombia

Joshua Tapias
Robótica (RBTC)
Escuela Colombiana de Ingeniería Julio Garavito, Bogotá, Colombia

Profesor: Daniel Palomino
Escuela Colombiana de Ingeniería Julio Garavito, Bogotá, Colombia
28 de octubre de 2026

***Resumen*—Se construyeron en un workspace local los paquetes de simulación del UR5 con gripper paralelo del repositorio `ur5_gripper_packages` y se verificó el control del brazo y del efector final mediante comandos publicados en ROS 2. Con MoveIt Setup Assistant se generó un paquete de planificación de trayectorias que, tras cuatro correcciones manuales, se conectó a la simulación de Gazebo. Se desarrollaron dos paquetes propios: un nodo que ejecuta una rutina Pick and Place entre tres puntos de interés comandando directamente los controladores de `ros2_control`, y un nodo basado en `MoveGroupInterface` que planifica y ejecuta la misma tarea dentro del ecosistema de MoveIt, con objetivos articulares, cartesianos y rectilíneos y una escena de planificación con objetos de colisión. El nodo de MoveIt completó las nueve etapas de la rutina en cuatro de cuatro ejecuciones con la escena normal, con planes calculados en 14 a 61 ms y trayectorias cartesianas resueltas al 100 %, y rodeó un pilar interpuesto en el camino. Se documentan once dificultades, su diagnóstico y las alternativas consideradas; la limitación de memoria del entorno WSL2 y los procesos huérfanos del puente de reloj fueron determinantes en varias de ellas.**

***Palabras clave*—ROS 2, UR5, MoveIt 2, Gazebo, ros2_control, Pick and Place, planificación de trayectorias.**

I. Objetivo

Construir en un workspace local los paquetes de simulación del UR5 con gripper paralelo del repositorio `ur5_gripper_packages` [1], verificar el control del brazo y del efector final mediante comandos publicados en ROS 2, generar un paquete de planificación de trayectorias con MoveIt Setup Assistant [5], desarrollar un paquete propio con un nodo que simule una rutina Pick and Place entre al menos tres puntos de interés y, finalmente, utilizar la configuración de MoveIt para ejecutar trayectorias desde un nodo que use el ecosistema de MoveIt.

II. Entorno de trabajo

La Tabla I resume el entorno. La memoria asignada a WSL resultó ser el factor limitante de toda la práctica: ejecutar Gazebo, RViz y `move_group` de forma simultánea dejó el entorno gráfico alrededor de 9 fps, con un factor de tiempo real sensiblemente inferior a 1. Varias de las dificultades de la Sección X son consecuencia directa de esa limitación.

La actividad 4 se ejecutó y midió en un segundo entorno, también descrito en la Tabla I: un contenedor Linux sin GPU, con ROS 2 Kilted instalado desde RoboStack (conda) y RViz renderizado por software sobre un servidor X virtual. Con Gazebo, `move_group` y RViz en marcha, el factor de tiempo real medido sobre `/clock` fue de 0,43, del mismo orden que el observado en WSL2.

TABLA I
Entorno de trabajo

Elemento	Act. 1 a 3	Act. 4
Sistema	Ubuntu 24.04.4 LTS sobre WSL2	Ubuntu 24.04.5, contenedor
ROS 2	Kilted (apt)	Kilted (RoboStack)
Simulador	Gazebo + `gz_ros2_control`	Gazebo Ionic 9.5, `gz_ros2_control` 2.0.17
MoveIt	2	2.14.3
Núcleos	16	2
Memoria	3,7 GiB (1,9 libres)	7,7 GiB
GPU	No usada	No (render por software)
Factor tiempo real	< 1 ($\approx$9 fps)	0,43

III. Enlaces a los entregables

Los entregables se listan en la Tabla II. Los videos de las actividades 1 a 3 se publicaron como *release* del repositorio del nodo de Pick and Place: `actividad_1` muestra el control del brazo y del gripper por tópicos; `actividad_2`, la planificación de trayectorias con MoveIt contra la simulación de Gazebo; y `actividad_3`, la rutina Pick and Place por tópicos y acciones. El repositorio `lab_robotica-` reúne la actividad 4: el nodo, la configuración de MoveIt con la que se ejecutó, el video `actividad_4`, los registros y datos de las ejecuciones y este informe.

IV. Construcción de los paquetes de simulación

Los paquetes del repositorio base se clonaron en `~/ur5_ws/src` y se compilaron con `colcon`. El repositorio contiene dos paquetes:

- `ur5_robotiq_description` (`ament_cmake`): descripción URDF/XACRO del UR5 con un gripper paralelo de dos dedos prismáticos y declaración del bloque `ros2_control` con el plugin `gz_ros2_control/GazeboSimSystem` [8].

TABLA II
ENLACES A LOS ENTREGABLES

Entregable	Enlace
Paquete con el nodo de Pick and Place (actividad 3)	https://github.com/Samuel-morenom/ur5_pick_and_place
Paquete de planificación de trayectorias (MoveIt Setup Assistant)	https://github.com/Samuel-morenom/ur5_robotiq_moveit
Repositorio de la actividad 4 e informe	https://github.com/tapiasjoshua/lab_robotica-
Nodo con el ecosistema de MoveIt (actividad 4)	https://github.com/tapiasjoshua/lab_robotica-/tree/main/ur5_moveit_trajectory
Configuración de MoveIt usada en la actividad 4	https://github.com/tapiasjoshua/lab_robotica-/tree/main/ur5_robotiq_moveit
Videos de las actividades 1 a 3	https://github.com/Samuel-morenom/ur5_pick_and_place/releases/tag/v1.0
Video de la actividad 4	https://github.com/tapiasjoshua/lab_robotica-/blob/main/videos/actividad_4.mp4
Registros y datos de las ejecuciones	https://github.com/tapiasjoshua/lab_robotica-/tree/main/resultados

- `ur5_robotiq_bringup` (`ament_python`): lanzadores de Gazebo y RViz, configuración de controladores y mundo de simulación.

La descripción incluye `ur_description/urdf/ur_macro.xacro` y los parámetros del UR5, de modo que `ur_description` [2] debe estar instalado. El controlador del brazo es del tipo `ur_controllers/ScaledJointTrajectoryController`, del driver oficial de Universal Robots [3]. El paquete de descripción declara además `robotiq_description` como dependencia de ejecución, aunque la descripción empleada (`simple_gripper.xacro`) es propia del repositorio y no la utiliza; la resolución de dependencias se completó omitiendo esa clave:

```
rosdep install --from-paths src --ignore-src
   -r -y --skip-keys robotiq_description
```

V. CONTROL DEL ROBOT MEDIANTE COMANDOS PUBLICADOS

A. Identificación de las interfaces

Con la simulación en marcha, `ros2 topic list` y `ros2 control list_controllers` permitieron identificar los controladores activos [4]:

- `joint_state_broadcaster`: publica el estado articular en `/joint_states`.
- `scaled_joint_trajectory_controller` (`ur_controllers`): acepta trayectorias por el tópico `joint_trajectory` y por la acción `follow_joint_trajectory`.
- `gripper_controller`, de tipo `position_controllers/JointGroupPositionController`: acepta posiciones por el tópico `commands`.

B. Movimiento del brazo y del gripper

El brazo se comanda con un `trajectory_msgs/msg/JointTrajectory` con los seis nombres de articulación y al menos un punto con posiciones y `time_from_start`. El gripper no expone una acción: se comanda con un `std_msgs/msg/Float64MultiArray` de dos elementos, uno por dedo, en metros (Fig. 1). Los límites del par prismático en `simple_gripper.xacro` son [0,002; 0,042] m, de modo que ambos valores quedan dentro del rango útil sin forzar los topes.

terminal — actividad 1

```
# Brazo: un punto de trayectoria con las seis juntas
ros2 topic pub --once \
  /scaled_joint_trajectory_controller/joint_trajectory \
  trajectory_msgs/msg/JointTrajectory \
  "{joint_names: ['shoulder_pan_joint','shoulder_lift_joint',
    'elbow_joint','wrist_1_joint','wrist_2_joint','wrist_3_joint'],
    points: [{positions: [0.0,-1.57,1.57,-1.57,-1.57,0.0],
              time_from_start: {sec: 3}}]}"

# Gripper: posicion por dedo [m], limites [0.002, 0.042]
ros2 topic pub --once /gripper_controller/commands \
  std_msgs/msg/Float64MultiArray "{data: [0.04, 0.04]}"   # abre
ros2 topic pub --once /gripper_controller/commands \
  std_msgs/msg/Float64MultiArray "{data: [0.004, 0.004]}" # cierra
```

Fig. 1. Comandos publicados para mover el brazo y abrir/cerrar el gripper.

TABLA III
CONFIGURACIÓN DEL PAQUETE GENERADO CON MOVEIT SETUP ASSISTANT

Pestaña	Configuración
Self-Collisions	Matriz generada con valores por defecto
Virtual Joints	`world_joint`, fija, `world` → `base_link_parent`
Planning Groups	`ur_arm`: cadena `base_link`–`tool0`, `kdl_kinematics_plugin`. `gripper`: `gripper_finger_joint` y `right_finger_joint`, sin solver
Robot Poses	`ur_arm`: Home, Ready. `gripper`: Open (0,04; 0,04), Close (0,004; 0,004)
End Effectors	`arm_eef`, grupo `gripper`, link padre `tool0`
ros2_control	`scaled_joint_trajectory_controller` y `gripper_controller`
MoveIt Controllers	`scaled_joint_trajectory_controller` (FollowJointTrajectory, por defecto)

Para la evidencia audiovisual se preparó el script `demo_actividad1.sh`, que lista los controladores activos, muestra los tópicos de control y el tipo de mensaje del gripper, y ejecuta una secuencia de movimientos del brazo intercalados con aperturas y cierres del efector, imprimiendo el vector de posiciones publicado en cada paso.

VI. PAQUETE DE PLANIFICACIÓN DE TRAYECTORIAS

Siguiendo el instructivo `MoveitSetupAssConfig.md` del repositorio base se generó el paquete `ur5_robotiq_moveit` a partir de `ur_with_gripper_moveit.urdf.xacro`. La Tabla III documenta la configuración obtenida.

TABLA IV
POSES DE LA RUTINA (RAD) Y ALTURA DE LA PUNTA (M)

Punto	pan	lift	elbow	Altura
HOME	0,000	−1,5708	1,5708	0,35
P1_APPROACH	0,000	−1,300	1,600	0,22
P1_PICK	0,000	−1,020	1,650	0,06
P2_TRANSFER	−0,785	−1,450	1,300	0,41
P3_APPROACH	−1,571	−1,300	1,600	0,22
P3_PLACE	−1,571	−1,020	1,650	0,06

La salida directa del asistente no era funcional y requirió cuatro correcciones manuales, documentadas en la Sección X: completar las listas de juntas (Fig. 2), declarar límites de aceleración (Fig. 3), retirar el `gripper_controller` de MoveIt y fijar `use_sim_time`. El paquete se utilizó conectado a la simulación de Gazebo en lugar de a su propio `demo.launch.py`, por las razones de la dificultad 3.

Con esa configuración se verificó la planificación y ejecución de trayectorias del grupo `ur_arm`: el usuario desplaza el marcador interactivo en RViz, MoveIt calcula la trayectoria y el brazo la ejecuta en Gazebo. También se verificó la planificación hacia las poses predefinidas Home y Ready. El grupo `gripper` es planificable, pero no ejecutable contra la simulación (dificultad 6).

Para ejecutar la actividad 4 en el segundo entorno se reprodujo el paquete con la misma estructura que genera el asistente (SRDF, `kinematics.yaml`, `joint_limits.yaml`, `moveit_controllers.yaml`, lanzadores y `.setup_assistant`) y con las cuatro correcciones ya aplicadas, más un `ompl_planning.yaml` con una verificación de colisiones más fina para `ur_arm` (dificultad 11). La matriz de autocolisiones se generó con `collisions_updater`, la herramienta de línea de comandos que usa el mismo algoritmo de la pestaña *Self-Collisions* (10 000 muestras): deshabilitó 19 pares, 9 adyacentes y 10 que nunca colisionan. Como el URDF de MoveIt ya tiene `world` como link raíz, la junta virtual se declaró con `world` como link hijo. Esta copia está publicada junto al nodo en `lab_robotica-`.

VII. PAQUETE PROPIO: RUTINA PICK AND PLACE

Se desarrolló el paquete `ur5_pick_and_place` (`ament_python`), con dos nodos. No depende de MoveIt: comanda directamente los controladores de `ros2_control`.

A. Generación de poses sin cinemática inversa

Las poses se construyen con la relación

$$w_1 = -(\theta_{lift} + \theta_{elbow}) - \frac{\pi}{2}, \quad w_2 = -\frac{\pi}{2}, \quad w_3 = 0 \qquad (1)$$

que mantiene el eje Z de `tool0` vertical hacia el piso para cualquier valor de `shoulder_pan`. Con cinemática directa y los parámetros DH del UR5 ($a_2 = -0{,}425$, $a_3 = -0{,}39225$, $d_1 = 0{,}089159$, $d_4 = 0{,}10915$, $d_5 = 0{,}09465$, $d_6 = 0{,}0823$) se obtienen las alturas de la punta de los dedos de la Tabla IV.

En el plano horizontal los tres puntos de operación quedan bien separados: P1 en $(-0{,}634; -0{,}109)$, P2 en $(-0{,}455; 0{,}300)$ y P3 en $(-0{,}109; 0{,}634)$ m. Ninguna pose interseca el piso y todas quedan dentro del espacio de trabajo: el radio máximo empleado es 0,64 m frente a los 0,85 m de alcance nominal.

TABLA V
INTERFACES DEL NODO DE PICK AND PLACE

Función	Interfaz
Brazo	Acción `control_msgs/FollowJointTrajectory` en `/scaled_joint_trajectory_controller/follow_joint_trajectory`
Brazo (alterno)	Tópico `trajectory_msgs/JointTrajectory` en `.../joint_trajectory`
Gripper	Tópico `std_msgs/Float64MultiArray` en `/gripper_controller/commands`
Entrada digital	Tópico `std_msgs/Bool` en `/digital_input_1` y servicio `std_srvs/SetBool` en `/set_digital_input`
Salida digital	Tópico `std_msgs/Bool` en `/digital_output_1` (QoS *transient local*)
Mensajes	Tópico `std_msgs/String` en `/pick_and_place/status`

B. Secuencia ejecutada

1) HOME y apertura del gripper.
2) **P1**: `P1_APPROACH` y `P1_PICK`, cierre del gripper, activación de la salida digital DO1 y mensaje informativo de pieza sujetada.
3) Retiro y traslado a **P2**: mensaje informativo de inspección y espera de la entrada digital DI1.
4) **P3**: `P3_APPROACH` y `P3_PLACE`, apertura del gripper, desactivación de DO1 y mensaje de pieza liberada.
5) Retorno a HOME.

La rutina cubre las cinco operaciones solicitadas: apertura y cierre del gripper, mensajes informativos, espera de entrada digital y activación de salida digital.

C. Interfaces del nodo

La Tabla V resume las interfaces. Se eligió la acción `FollowJointTrajectory` como mecanismo principal porque confirma la finalización de cada movimiento; la publicación directa en el tópico, disponible con `use_action:=false`, no entrega retroalimentación y obliga a esperar por tiempo fijo. La entrada digital se expuso como tópico y como servicio para ilustrar ambos mecanismos de simulación de señales de planta. Para no bloquear los callbacks, el nodo gira un `MultiThreadedExecutor` en un hilo secundario mientras la rutina se ejecuta en el hilo principal con esperas bloqueantes sobre los *futures* de la acción.

D. Nodo auxiliar de entradas y salidas digitales

El nodo `digital_io_sim` simula el panel de señales de planta: reporta en consola el estado de DO1, equivalente a una lámpara o al electroimán del efector, y publica un pulso en DI1 cuando se invoca el servicio `/trigger_digital_input` o cuando vence el temporizador configurable `auto_confirm_delay`.

ur5_robotiq_moveit/config/moveit_controllers.yaml

```yaml
# config/moveit_controllers.yaml (corregido)
moveit_controller_manager: moveit_simple_controller_manager/MoveItSimpleControllerManager
moveit_simple_controller_manager:
  controller_names:
    - scaled_joint_trajectory_controller   # gripper_controller retirado
  scaled_joint_trajectory_controller:
    type: FollowJointTrajectory
    action_ns: follow_joint_trajectory
    default: true
    joints:          # el asistente habia generado joints: []
      - shoulder_pan_joint
      - shoulder_lift_joint
      - elbow_joint
      - wrist_1_joint
      - wrist_2_joint
      - wrist_3_joint
```

Fig. 2. `moveit_controllers.yaml` tras las correcciones de las dificultades 2 y 6.

ur5_robotiq_moveit/config/joint_limits.yaml

```yaml
# config/joint_limits.yaml (fragmento corregido)
default_velocity_scaling_factor: 0.5
default_acceleration_scaling_factor: 0.5
joint_limits:
  shoulder_pan_joint:
    has_velocity_limits: true
    max_velocity: 3.15
    has_acceleration_limits: true   # false en la salida del asistente
    max_acceleration: 5.0
  wrist_1_joint:
    has_velocity_limits: true
    max_velocity: 3.20
    has_acceleration_limits: true
    max_acceleration: 5.0
  gripper_finger_joint:
    has_velocity_limits: true
    max_velocity: 0.1              # valor del XACRO
    has_acceleration_limits: true
    max_acceleration: 5.0
```

Fig. 3. Fragmento de `joint_limits.yaml` con límites de aceleración (dificultad 4).

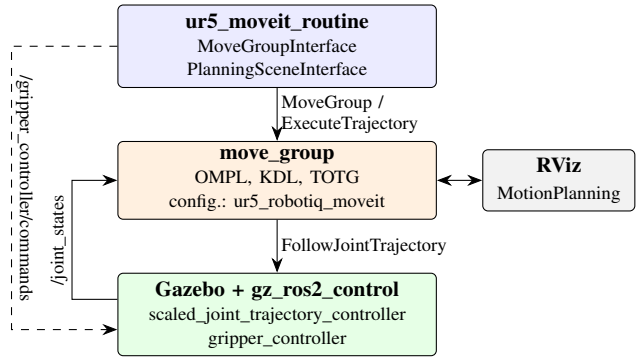


Fig. 4. Arquitectura de la actividad 4. El gripper se comanda por tópico (línea discontinua) porque su controlador no expone acción (dificultad 6).

VIII. Ejecución de trayectorias desde un nodo con el ecosistema de MoveIt

Para la cuarta actividad se desarrolló el paquete `ur5_moveit_trajectory` (`ament_cmake`, C++), que reutiliza la configuración de la Sección VI y ejecuta la misma tarea de Pick and Place, pero dejando que MoveIt planifique cada movimiento con la interfaz `MoveGroupInterface` [9]. La Fig. 4 muestra la arquitectura.

ur5_moveit_trajectory/launch/move_group_sim.launch.py

```python
moveit_config = (
    MoveItConfigsBuilder("ur", package_name="ur5_robotiq_moveit")
    .robot_description(file_path=urdf)   # ur_with_gripper_moveit
    .trajectory_execution(file_path="config/moveit_controllers.yaml")
    .planning_pipelines(pipelines=["ompl"])
    .to_moveit_configs())

move_group = Node(
    package="moveit_ros_move_group", executable="move_group",
    parameters=[
        moveit_config.to_dict(),
        {"use_sim_time": True},                          # dif. 5
        {"trajectory_execution."
         "execution_duration_monitoring": False,         # dif. 7
         "trajectory_execution.allowed_start_tolerance": 0.05},
    ])
```

Fig. 5. Parámetros de `move_group` en el lanzador propio.

A. Lanzador de move_group contra la simulación

El lanzador `move_group_sim.launch.py` sustituye al `demo.launch.py` del asistente y deja fijas, en lugar de aplicarlas en caliente (Fig. 11), las correcciones de las dificultades 5, 7 y 8 (Fig. 5). Se construye la configuración con `MoveItConfigsBuilder` usando el mismo URDF con el que se generó el SRDF, sin el link `ground_plane` de la simulación; el suelo pasa a ser un objeto de colisión que agrega el nodo.

B. Funcionamiento del nodo

El nodo `ur5_moveit_routine` gira un `SingleThreadedExecutor` en un hilo secundario, necesario para que `MoveGroupInterface` reciba el estado del robot, y ejecuta la rutina en el hilo principal (Fig. 6). Emplea cuatro tipos de objetivo:

- **Pose con nombre** del SRDF (`Home`) al inicio y al final.
- **Objetivo cartesiano** sobre P1 y P3, resuelto con `setJointValueTarget(pose)`: la cinemática inversa de KDL se siembra con el estado actual, lo que evita saltos a otra rama de la solución.
- **Objetivo articular** para el traslado a P2, con la relación (1).
- **Trayectorias rectilíneas** con `computeCartesianPath` para descender y retirarse en P1 y P3, exigiendo que se cubra al menos el 95 % del camino (Fig. 7).

Las poses de toma y depósito no se escriben a mano: se calculan por cinemática directa con el `RobotState` de MoveIt a partir de las articulaciones `P1_PICK` y `P3_PLACE` de la Tabla IV, y la aproximación se sitúa 15 cm sobre cada una (Fig. 8).

C. Escena de planificación

Con `PlanningSceneInterface` el nodo agrega el suelo (cara superior en $z = -0{,}01$ m, igual que `ground_plane`), dos estaciones de 5 cm de altura bajo P1 y P3 y una pieza cúbica de 3 cm entre los dedos. Al cerrar el gripper, la pieza se adjunta a `tool0` con `attachObject`, declarando los dedos y la base del gripper como *touch links*, de modo que MoveIt la considera en la verificación de colisiones durante el traslado y la suelta sobre la estación de P3 con `detachObject`. El parámetro `add_obstacle:=true` agrega un pilar de 0,40 m en el punto medio entre P2 y P3, sobre el camino directo de la herramienta.

D. Tratamiento de los falsos fallos de control

Con factor de tiempo real inferior a 1, MoveIt puede reportar `TIMED_OUT` o `CONTROL_FAILED` aunque el robot llegue (dificultad 7). Además de desactivar el monitor de duración, el nodo compara `/joint_states` con el último punto de la trayectoria, con tolerancia de 0,02 rad, antes de declarar un fallo; si el robot llegó, registra una advertencia y continúa. Además, antes de cada planificación el nodo espera a que el brazo esté quieto (variación menor de 1 mrad en tres lecturas de `/joint_states`) y, si MoveIt rechaza una trayectoria cartesiana, la recalcula una vez desde el estado real (dificultad 10).

E. Procedimiento y resultados

La Fig. 9 muestra el orden de arranque, que incorpora la lección de la dificultad 9: la simulación se lanza sola y el resto de los nodos solo cuando existe `/joint_states`. El progreso se publica en `/moveit_routine/status`.

Con un único publicador en `/clock`, la rutina se ejecutó cuatro veces seguidas sobre la misma simulación con la escena normal. Las cuatro completaron las nueve etapas sin advertencias ni reintentos. Durante las corridas se grabó un *rosbag* con `/joint_states`, `/display_planned_path`, `/gripper_controller/commands`, `/moveit_routine/status` y `/clock`; la Tabla VI resume lo medido y la Fig. 10 muestra las articulaciones durante la primera corrida.

Observaciones. El error articular final es prácticamente nulo porque la interfaz de posición de `gz_ros2_control` impone la posición comandada en cada paso de la simulación; no representa la dinámica de un UR5 real, de modo que este resultado valida la cadena MoveIt–controlador y no el seguimiento físico. La primera corrida dura 1,6 s más que las demás porque parte de la configuración de ceros; en las siguientes el robot ya está en `Home` y MoveIt devuelve un plan de un solo punto. En la Fig. 10 el dedo izquierdo baja momentáneamente a su límite inferior (2 mm) al cerrar y queda 2 mm por encima del comando tras la segunda apertura, mientras el derecho sigue el comando; la diferencia aparece solo en la física del dedo y no afecta la rutina, pero indica que los dedos deberían acoplarse con una junta *mimic*.

TABLA VI
RESULTADOS DE LA ACTIVIDAD 4 (CUATRO CORRIDAS, ESCENA NORMAL)

Medida	Valor
Corridas completas (9/9 etapas)	4 de 4
Trayectorias ejecutadas	36 (9 por corrida)
Tiempo de planificación (20 planes)	14–61 ms, mediana 37 ms
Camino cubierto por `computeCartesianPath`	100 % en las 16 trayectorias
Duración de los planes	0,74 s (descensos), 0,79–1,12 s (aproximaciones), 1,63 s (Home)
Error articular final máximo	$5{,}5 \times 10^{-8}$ rad
Duración de la rutina (tiempo simulado)	15,6 s (1.ª), 13,9–14,0 s (resto)
Duración de la rutina (reloj de pared)	22,9–29,5 s, con arranque del nodo
Falsos fallos de control / reintentos	0 / 0

TABLA VII
EFECTO DEL PILAR EN LOS PLANES DE OMPL

Plan	Escena normal	Con pilar
Aproximación a P3	13 puntos, 1,12 s	24 puntos, 2,21 s
Retorno a Home	18 puntos, 1,63 s	56 puntos, 5,44 s
Tiempo de planificación	14–61 ms	458–556 ms

Prueba con obstáculo. Con `add_obstacle:=true` se agrega un pilar de $0{,}08 \times 0{,}08 \times 0{,}40$ m en el punto medio entre P2 y P3. La primera prueba falló al planificar la aproximación a P3 (dificultad 11); con la verificación de colisiones más fina, la rutina completó las nueve etapas y MoveIt rodeó el pilar. La Tabla VII compara los planes afectados con la escena normal. El precio de la verificación más fina es el tiempo de planificación, que pasa de decenas de milisegundos a cerca de medio segundo por plan.

El video `actividad_4` (92 s) muestra en RViz, con las etapas sobreimpresas, una corrida normal y la corrida con el pilar, en tiempo de pared. RViz representa el estado que publica la simulación en `/joint_states`.

IX. HALLAZGOS SOBRE LOS PAQUETES BASE

Altura de aparición en Gazebo. En `sim_gz_sgb.launch.py` los argumentos de `ros_gz_sim create` incluyen `"z", "0.1"` sin el guion (`-z`), por lo que la posición no se aplica. No afecta la simulación porque el URDF ancla `base_link_parent` al frame `world` con una junta fija.

Plano del piso sin espesor. El link `ground_plane` declara una caja de $5 \times 5 \times 0$. Una colisión de espesor nulo puede ser ignorada o generar contactos inestables en el motor de física.

Mundo por defecto. El argumento `world_file` vale `empty.sdf`, que Gazebo resuelve contra sus mundos internos y no contra `worlds/empty.sdf` del paquete. La escena no

src/moveit_routine_node.cpp — run()

```cpp
const Pose pick  = forwardKinematics(verticalToolPose(0.000, -1.020, 1.650));
const Pose place = forwardKinematics(verticalToolPose(-1.571, -1.020, 1.650));
const auto transfer_q = verticalToolPose(-0.785, -1.450, 1.300);
const Pose transfer = forwardKinematics(transfer_q);

moveToNamed(home_state_);                 // pose con nombre del SRDF
buildScene(pick, transfer, place);        // suelo, estaciones, pieza (y pilar)
openGripper();

moveToPose(raised(pick, approach_height_));   // P1: objetivo cartesiano + IK
moveCartesian(pick);                          // descenso rectilineo
closeGripper();  attachPart();
moveCartesian(raised(pick, approach_height_));

moveToJoints(transfer_q);                     // P2: objetivo articular

moveToPose(raised(place, approach_height_)); // P3
moveCartesian(place);
openGripper();   detachPart();
moveCartesian(raised(place, approach_height_));
moveToNamed(home_state_);
```

Fig. 6. Secuencia principal del nodo `ur5_moveit_routine` (manejo de errores omitido).

src/moveit_routine_node.cpp — movimientos

```cpp
bool moveToPose(const Pose& target) {
  // IK sembrada con el estado actual: evita saltos de rama
  if (!arm_.setJointValueTarget(target, eef_link_))
    arm_.setPoseTarget(target, eef_link_);
  return planAndExecute("objetivo cartesiano");
}

bool moveCartesian(const Pose& target) {
  for (int attempt = 1; attempt <= 2; ++attempt) {  // dificultad 10
    waitUntilSettled();                 // brazo quieto antes de calcular
    arm_.setStartStateToCurrentState();
    moveit_msgs::msg::RobotTrajectory traj;
    const double fraction =
        arm_.computeCartesianPath({target}, eef_step_, traj);
    if (fraction < min_fraction_) return false;
    if (execute(traj, "trayectoria cartesiana")) return true;
  }
  return false;
}

bool execute(const RobotTrajectory& traj, const std::string& label) {
  const auto code = arm_.execute(traj);
  if (code == MoveItErrorCode::SUCCESS) return true;
  // Dificultad 7: verificar contra /joint_states antes de fallar
  const bool timing = code.val == MoveItErrorCodes::TIMED_OUT ||
                      code.val == MoveItErrorCodes::CONTROL_FAILED;
  return timing && reachedEnd(traj);
}
```

Fig. 7. Objetivo cartesiano, trayectoria rectilínea y validación de la ejecución (fragmento simplificado).

src/moveit_routine_node.cpp — modelo y escena

```cpp
Pose forwardKinematics(const std::vector<double>& q) {
  auto state = arm_.getCurrentState(5.0);
  const auto* jmg = state->getJointModelGroup(group_name_);
  state->setJointGroupPositions(jmg, q);
  state->update();
  return isometryToPose(state->getGlobalLinkTransform(eef_link_));
}

void attachPart() {
  const std::vector<std::string> touch{
      "gripper_base_link", "left_finger_link", "right_finger_link"};
  arm_.attachObject("pieza", eef_link_, touch);
}

// Herramienta vertical hacia el piso para cualquier shoulder_pan
std::vector<double> verticalToolPose(double pan, double lift, double elbow) {
  return {pan, lift, elbow, -(lift + elbow) - M_PI / 2, -M_PI / 2, 0.0};
}
```

Fig. 8. Cinemática directa con el modelo de MoveIt y adjunción de la pieza.

tiene mesa ni objetos, de modo que los puntos de toma y depósito son coordenadas del espacio de trabajo.

terminal — actividad 4

```
# Terminal 1: simulacion sola, esperar /joint_states
ros2 launch ur5_robotiq_bringup sim_gz_sgb.launch.py launch_rviz:=false

# Terminal 2: move_group + RViz contra Gazebo
ros2 launch ur5_moveit_trajectory move_group_sim.launch.py

# Terminal 3: rutina (add_obstacle:=true agrega un pilar)
ros2 launch ur5_moveit_trajectory moveit_routine.launch.py
ros2 topic echo /moveit_routine/status
```

Fig. 9. Orden de arranque para la actividad 4.

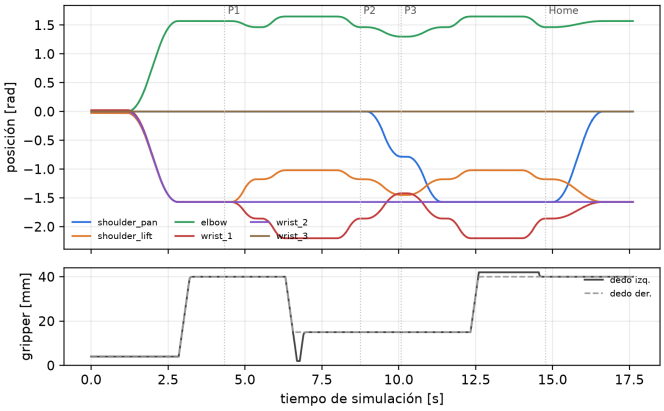


Fig. 10. Posiciones articulares y de los dedos registradas en `/joint_states` durante la primera corrida. Las líneas punteadas marcan el inicio de cada etapa.

Divergencia entre instructivo y configuración. `MoveitSetupAssConfig.md` indica configurar el `gripper_controller` como `GripperActionController`, mientras que `controllers_sg.yaml` lo declara como `JointGroupPositionController`. Es el origen de la dificultad 6.

X. Dificultades encontradas y alternativas consideradas

A. El asistente rechaza tool0 como link padre del efector

Síntoma: al guardar *End Effectors*: "The specified parent group 'ur_arm' must contain the specified parent link 'tool0' ". *Causa:* el instructivo define `ur_arm` como lista de juntas; un grupo así solo contiene los links que cuelgan de esas seis juntas, y `tool0` se une a `wrist_3_link` con juntas fijas que no forman parte del grupo. *Alternativas:* dejar vacío el campo opcional *Parent Group*; se descartó en favor de la segunda opción. *Resolución:* se redefinió `ur_arm` como cadena cinemática de `base_link` a `tool0`, lo que incorpora los links fijos del extremo y hace que la cinemática inversa se resuelva hasta la brida de la herramienta.

B. El paquete generado no arranca por listas vacías

Síntoma: "Expected 'value' to be one of [float, int, str, bool, bytes], but got '()' of type tuple". *Causa:* el asistente generó `joints: []` en `moveit_controllers.yaml` y `command_interfaces: []` y `state_interfaces: []` en `ros2_controllers.yaml`. *Fuentes: issues* #2837 [6] y #3165 [7] de MoveIt 2; la solución es de la comunidad. *Resolución:* se completaron las cuatro listas (Fig. 2).

C. El controller_manager de demo.launch.py nunca inicializa

Síntoma: se repite "Waiting for data on 'robot_description' topic to finish initialization" y el *spawner* espera indefinidamente `/controller_manager/list_controllers`. *Diagnóstico:* `ros2 topic info /robot_description -v` mostró publicador y suscriptor con perfiles QoS compatibles (RELIABLE, TRANSIENT_LOCAL); el paquete generado no contiene ningún `*.urdf.xacro` ni `*.ros2_control.xacro`. *Causa:* la pestaña *ros2_control URDF Modifications* quedó con valores por defecto y el URDF base no declara bloque `ros2_control`: el `controller_manager` no encuentra hardware. *Alternativas:* regenerar el paquete; se descartó porque el asistente sobrescribe los YAML corregidos. *Resolución:* conectar MoveIt directamente a la simulación de Gazebo, que ya dispone de controladores reales; en la actividad 4 esto quedó formalizado en un lanzador propio.

D. La planificación falla en la parametrización temporal

Síntoma: "Response adapter 'AddTimeOptimalParameterization' failed to generate a trajectory". *Causa:* `joint_limits.yaml` con `has_acceleration_limits: false`; el algoritmo de parametrización temporal óptima requiere límites de velocidad y aceleración. *Observación:* el síntoma no apunta a la causa; la planificación geométrica era correcta y se descartaron antes hipótesis de colisión inicial y de límites del gripper. *Resolución:* velocidades reales del UR5 (3,15 rad/s en las juntas del brazo, 3,20 rad/s en las de muñeca), aceleraciones de 5,0 rad/s^2 y 0,1 m/s en los dedos (Fig. 3).

terminal — parámetros en caliente

```
# Dificultad 5: reloj de pared vs. tiempo simulado
ros2 param set /move_group use_sim_time true
ros2 param set /rviz use_sim_time true

# Dificultad 7: falsos CONTROL_FAILED con factor de tiempo real < 1
ros2 param set /move_group \
  trajectory_execution.execution_duration_monitoring false
```

Fig. 11. Correcciones aplicadas en caliente a `move_group` y RViz.

E. La ejecución se aborta por desincronización de reloj

Síntoma: "Didn't receive robot state (joint angles) with recent timestamp within 1.000000 seconds. Requested time 1793064201.08, but latest received state has time 31.617". *Causa:* `move_group` usa el reloj de pared, mientras `/joint_states` llega con tiempo de simulación; `use_sim_time:=true` no se propaga a los nodos en el lanzador generado. *Resolución:* parámetro fijado en caliente (Fig. 11); respaldo identificado: `trajectory_execution.allowed_start_tolerance 0.0`.

F. El grupo gripper es planificable pero no ejecutable

Síntoma: "Failed to send trajectory part 1 of 1 to controller gripper_controller" tras buscar `gripper_controller/follow_joint_trajectory`. *Causa:* el controlador de la simulación es un `JointGroupPositionController` sin acción. *Resolución:* se retiró de `moveit_controllers.yaml`; el efector se comanda por tópico. No se reemplazó el controlador de la simulación para no modificar el paquete base ni invalidar la actividad 1.

G. Falsos fallos de control por limitación de cómputo

Síntoma: "waitForExecution timed out" y `CONTROL_FAILED`, seguidos segundos después de "Controller 'scaled_joint_trajectory_controller' successfully finished". *Causa:* MoveIt compara la duración real con la estimada por un factor de tolerancia; con factor de tiempo real inferior a 1 el plazo se excede sistemáticamente. Es consecuencia directa de la memoria disponible (1,9 GiB, ≈9 fps). *Resolución:* monitor de duración desactivado y factores de escala elevados de 0,10 a 0,50; en la actividad 4, además, verificación del estado final en el propio nodo.

H. Colisión inicial con el plano del piso

Síntoma: el robot aparece en RViz en rojo, extendido y atravesando el piso, y la planificación se rechaza. *Causa:* `move_group` cargó el URDF de la simulación, que incluye `ground_plane`, ausente en el modelo con el que se generó el SRDF; en la configuración de ceros el brazo lo atraviesa. *Resolución:* llevar el brazo a HOME por tópico antes de abrir RViz. En la actividad 4 la causa se eliminó: `move_group` usa el URDF del SRDF y el suelo se agrega como objeto de colisión después de alcanzar `Home`.

I. Fallo intermitente de la aparición del robot

Síntoma: tras un cierre inesperado del sistema, Gazebo arranca pero no existen `/controller_manager` ni `/joint_states`. *Causa probable:* condición de carrera agravada por la carga; el `controller_manager` lo instancia `gz_ros2_control` cuando el robot entra en la escena, y si la aparición falla no se crea. *Resolución:* reiniciar la simulación aislada, esperar `/joint_states` y solo entonces arrancar el resto en orden; `gazebo_gui:=false` reduce el consumo durante las verificaciones.

J. Ejecuciones abortadas por relojes duplicados

Síntoma: en la actividad 4, dos corridas abortaron en etapas distintas: una con "Invalid Trajectory: start point deviates from current robot state more than 0.05 at joint 'shoulder_lift_joint' " en el descenso a P1, y otra con `PATH_TOLERANCE_VIOLATED` al ir a `Home`, con errores de posición de 0,58 rad en cuatro juntas y cientos de avisos "Detected jump back in time" en `move_group` y RViz. *Diagnóstico:* el *rosbag* mostró dos series de tiempo intercaladas en `/clock` (por ejemplo 12,344 s seguido de 11,778 s): 10 636 retrocesos en 127 185 mensajes. `ros2 topic info /clock` reportó cuatro publicadores. *Causa:* al detener lanzamientos anteriores de forma abrupta quedaron vivos tres procesos `parameter_bridge`. Todos republicaban el reloj de la simulación nueva, con retrasos distintos por la carga, y el tiempo parecía retroceder. El controlador evalúa la trayectoria con ese reloj, y MoveIt compara el estado con un instante equivocado. *Alternativas:* se atribuyó primero el fallo del descenso a que la trayectoria se calculaba antes de que el brazo se detuviera; la espera de reposo y el reintento se conservaron en el nodo porque son buena práctica, pero la corrida siguiente volvió a fallar, lo que descartó esa causa. *Resolución:* terminar los procesos huérfanos (`pkill -f parameter_bridge`) y verificar antes de cada prueba que `/clock` tenga un solo publicador. Con esa verificación, las cuatro corridas posteriores fueron exitosas. Es plausible que la dificultad 9 tenga el mismo origen, porque también apareció tras un cierre inesperado.

K. OMPL atraviesa un obstáculo delgado

Síntoma: con el pilar en la escena, la aproximación a P3 falla con "Computed path is not valid. Invalid states at index locations: [5 6 7] out of 13" y contactos entre `pilar` y `gripper_base_link`, `wrist_3_link` y `wrist_1_link`. *Causa:* OMPL verifica colisiones a lo largo de cada segmento del camino en pasos de `longest_valid_segment_fraction` por la extensión del espacio articular (0,005 por defecto). Con juntas de rango $\pm 2\pi$, ese paso permite que la muñeca avance varios centímetros entre verificaciones, de modo que un pilar de 8 cm queda entre dos estados revisados. El adaptador `ValidateSolution` revisa después la trayectoria interpolada, detecta el choque y rechaza el plan; MoveIt no llega a ejecutar una trayectoria en colisión. *Observación de desarrollo:* en una corrida anterior el pilar no se había agregado porque `add_obstacle: false` estaba en `routine.yaml`, y el valor específico del nodo prevalece sobre el argumento de lanzamiento; se retiró del YAML. *Resolución:* `ompl_planning.yaml` con `longest_valid_segment_fraction: 0.0005` para `ur_arm`, a costa de planes unas diez veces más lentos (Tabla VII).

XI. Conclusiones

Las cuatro actividades solicitadas se abordaron. El control por comandos publicados quedó verificado para el brazo y el efector final; el paquete de planificación se generó, se depuró y se demostró ejecutando movimientos sobre la simulación; el nodo de Pick and Place recorre tres puntos de interés ejecutando las cinco operaciones solicitadas; y el nodo con el ecosistema de MoveIt planifica y ejecuta la misma tarea con objetivos con nombre, articulares, cartesianos y rectilíneos sobre una escena con objetos de colisión: completó la rutina en cuatro de cuatro corridas con la escena normal y rodeó un pilar interpuesto en el camino.

El trabajo útil no estuvo en seguir el instructivo, sino en diagnosticar por qué no funcionaba. De las once dificultades, cuatro corresponden a archivos o lanzadores incompletos generados por el asistente (2 a 5), dos a divergencias entre el instructivo y la configuración real (1 y 6), dos a limitaciones de cómputo (7 y 9), una a la diferencia entre el modelo de la simulación y el del SRDF (8), una a procesos huérfanos que duplicaban el reloj de simulación (10) y una a la resolución con la que OMPL verifica colisiones (11). Ninguna era evidente a partir de su síntoma; el caso más ilustrativo es la dificultad 4, donde un fallo reportado como de planificación resultó ser de parametrización temporal, seguido de la dificultad 10, donde un error de tolerancia de trayectoria resultó ser un problema de reloj.

Implementar primero el nodo de Pick and Place sin MoveIt aisló el entregable principal de la cadena de problemas de configuración y funcionó desde la primera ejecución. La actividad 4 reutilizó ese diagnóstico: el lanzador propio fija las correcciones que antes se aplicaban en caliente y el nodo tolera los falsos fallos de ejecución. Queda pendiente ejecutar el efector final a través de MoveIt, que exigiría sustituir el `JointGroupPositionController` por un `GripperActionController`. Con más memoria asignada al entorno, la dificultad 7 probablemente no se presentaría, y verificar que `/clock` tenga un solo publicador antes de cada prueba evita la dificultad 10.

Referencias

[1] D. Palomino S., "ur5_gripper_packages," GitHub. [Online]. Available: https://github.com/DanielPalominoS/ur5_gripper_packages

[2] Universal Robots, "Universal_Robots_ROS2_Description," GitHub. [Online]. Available: https://github.com/UniversalRobots/Universal_Robots_ROS2_Description

[3] Universal Robots, "Universal_Robots_ROS2_Driver, paquete ur_controllers," GitHub. [Online]. Available: https://github.com/UniversalRobots/Universal_Robots_ROS2_Driver

[4] ros2_control Development Team, "ros2_control y ros2_controllers." [Online]. Available: https://control.ros.org

[5] PickNik Robotics, "MoveIt 2 documentation: MoveIt Setup Assistant." [Online]. Available: https://moveit.picknik.ai

[6] MoveIt 2, "MSA: controller joints cannot be empty list," issue #2837. [Online]. Available: https://github.com/moveit/moveit2/issues/2837

[7] MoveIt 2, “Generated launch files from setup assistant not working,” issue #3165. [Online]. Available: https://github.com/moveit/moveit2/issues/3165
[8] ros-controls, “gz_ros2_control,” GitHub. [Online]. Available: https://github.com/ros-controls/gz_ros2_control
[9] PickNik Robotics, “MoveIt 2 tutorials: Move Group C++ Interface.” [Online]. Available: https://moveit.picknik.ai/main/doc/examples/move_group_interface/move_group_interface_tutorial.html